

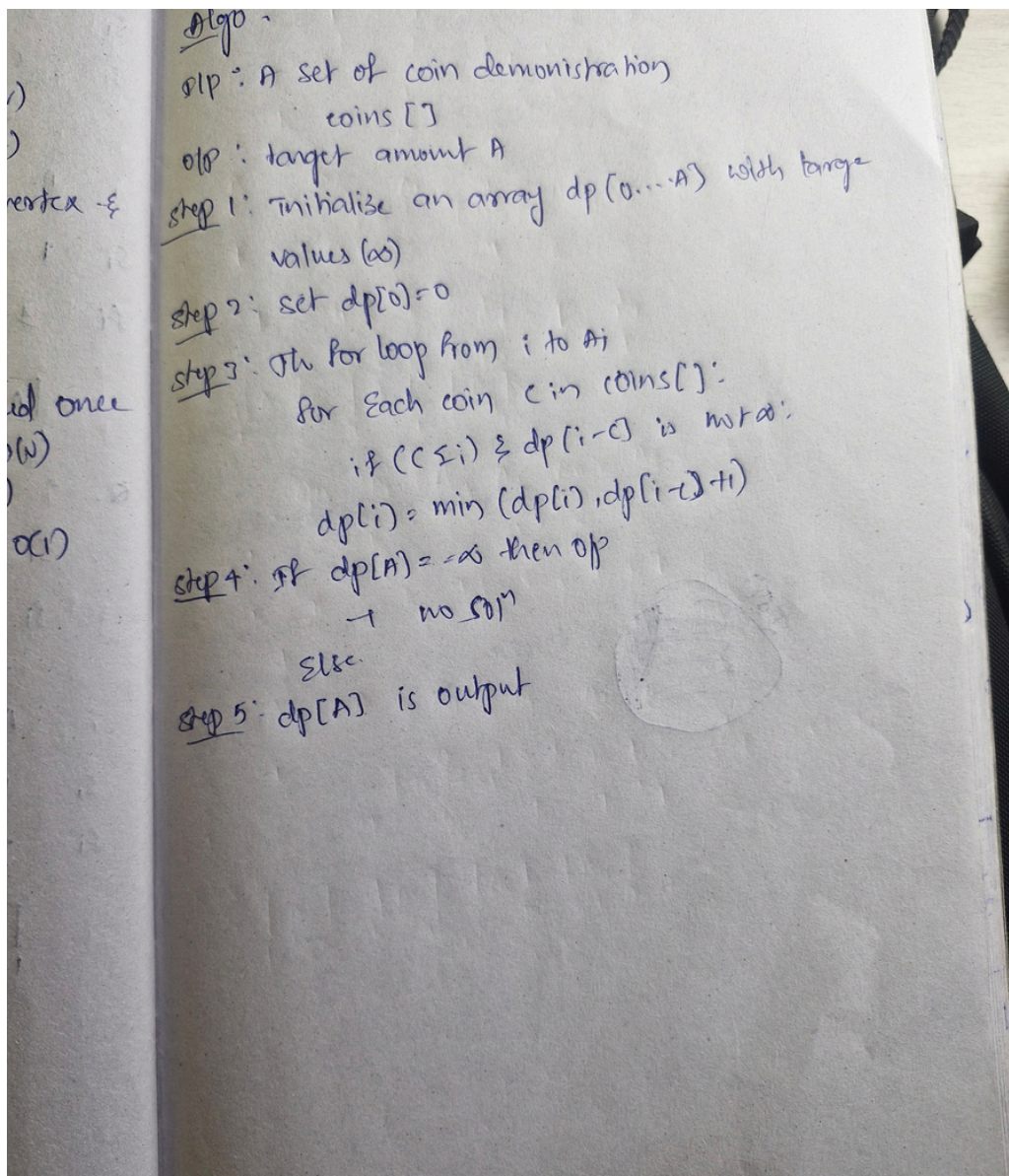
Practical: 7

Implementation of making a change problem using dynamic programming

20/09/2026

Aim: Implementation of making a change problem using dynamic programming

Algorithm:



Algo -

inp : A set of coin denomination
coins []

inp : target amount A

step 1: Initialize an array $dp[0 \dots A]$ with large values (∞)

step 2: set $dp[0] = 0$

step 3: For loop from i to A
For Each coin c in coins[]:
if $(c \leq i)$ & $dp[i-c]$ is not ∞ :
 $dp[i] = \min(dp[i], dp[i-c] + 1)$

step 4: If $dp[A] = \infty$ then op
→ no soln

else

step 5: $dp[A]$ is output

Program:

```
#include <bits/stdc++.h>

using namespace std;

//Function to compute minimum coins required
int coinChange(vector<int>&coins, int amount) {
    //DP array to store minimum coins for each value
    vector<int> dp(amount+1, INT_MAX);
    dp[0]=0; //base case: 0 coins needed for amount 0

    // Fill DP table
    for(int i=1; i<=amount; i++){
        for(int c : coins) {
            if(c<=i && dp[i-c]!=INT_MAX) {
                dp[i]=min(dp[i], dp[i-c] + 1);
            }
        }
    }

    return(dp[amount]==INT_MAX) ? -1 : dp[amount];
}

// Example usage
int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;

    int result = coinChange(coins, amount);
    if (result != -1)
        cout<<"Minimum coins required: " << result << endl;
```

else

cout << "No solution possible" << endl;

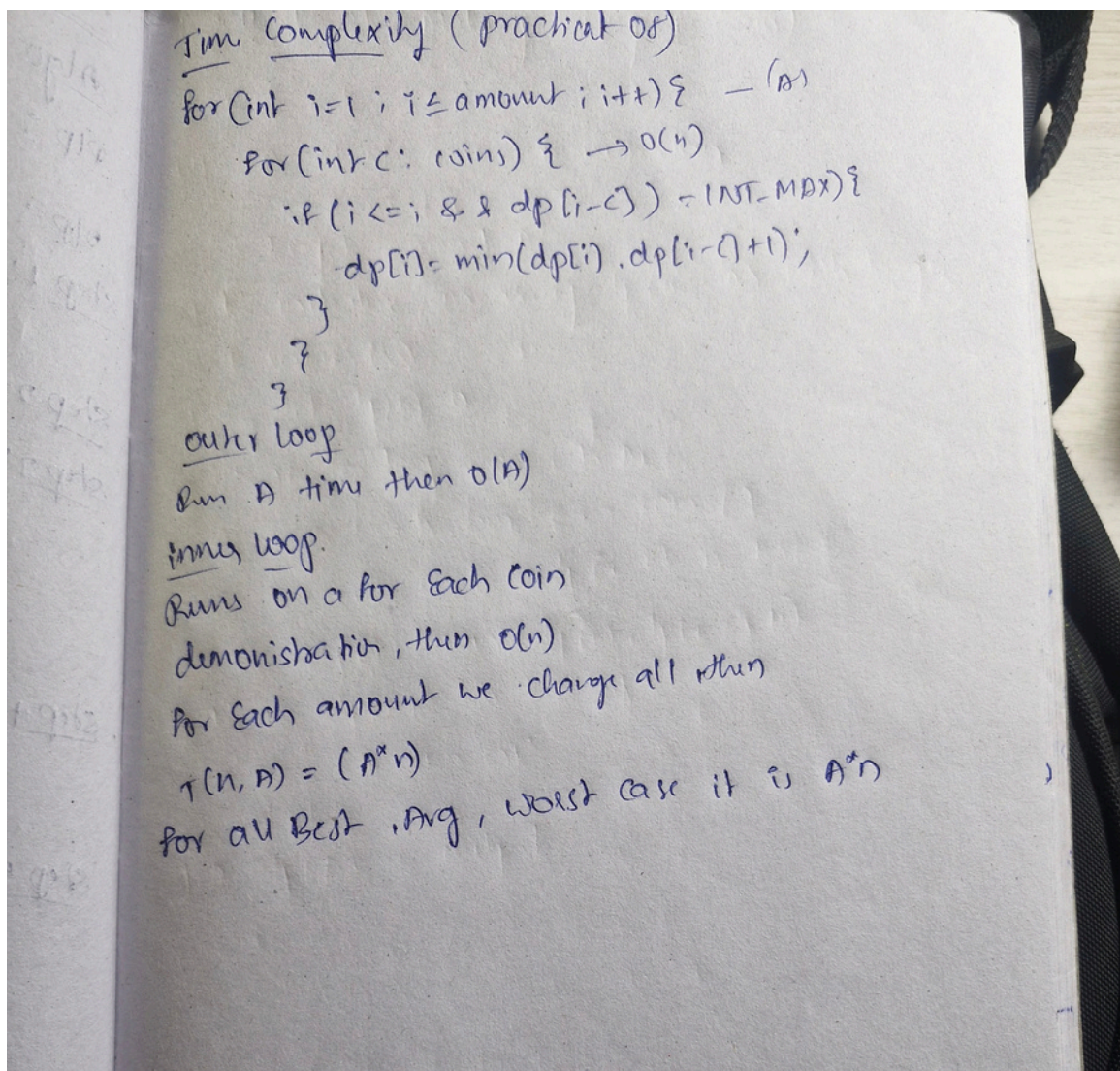
return 0;

}

Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ y.cpp -o y } ; if ($?) { .y }
Minimum coins required: 2
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



Conclusion:

Matrix Chain Multiplication and Coin Change both highlight how dynamic programming transforms exponential recursive solutions into efficient polynomial-time algorithms. In conclusion, DP ensures optimal subproblem reuse, yielding uniform complexity across best, average, and worst cases.